

Breadth-First Search (BFS) in Graphs

◆ Prompt

What is the Breadth-First Search (BFS) algorithm?

✓ Answer

- BFS is a **graph traversal algorithm** that explores vertices **level by level**.
- It starts at a source vertex s (level 0) and explores all **neighbors of s** first (level 1), then neighbors of neighbors (level 2), and so on.
- BFS uses a “**wavefront**” **approach**, imagining many explorers expanding outward simultaneously.

Key property: BFS finds the **shortest path (fewest edges)** from the source to any reachable vertex.

◆ Prompt

Explain the BFS process step by step.

✓ Answer

Initialize:

- Start at vertex s
- $\text{level}[0] = [s]$
- Mark s as visited

Repeat until no new vertices found:

- For each vertex u in the current level:
 - Check all edges (u, v)
 - If v is unvisited:
 - Mark v as visited
 - Add v to the **next level**
- Move to the **next level** and repeat

Terminate:

- When the next level is empty, all reachable vertices have been visited

Optional: Keep track of the **tree edges** that discovered each vertex to reconstruct BFS tree paths.

BFS Algorithm (Python)

```
def BFS(g, s, discovered):    """
    Perform BFS starting at vertex s in graph g.
    discovered: dictionary mapping each vertex to the edge that discovered
    it.
    """
    level = [s] # first level
    while len(level) > 0:
        next_level = []
        for u in level:
            for e in g.incident_edges(u): # iterate outgoing edges
                v = e.opposite(u)
                if v not in discovered:
                    discovered[v] = e # mark discovery edge
                    next_level.append(v)
        level = next_level # move to next level
```

Notes:

- BFS can also be implemented with a **FIFO queue** instead of explicit levels.
- Complexity: **$O(n + m)$** for adjacency list representation (n = vertices, m = edges).

◆ Prompt

What are the important properties of BFS?

✓ Answer

Visits all reachable vertices from the source s .

Shortest path property:

- For vertex v at level i , the path from s to v in the BFS tree has **exactly i edges**.
- Any other path from s to v has **at least i edges**.

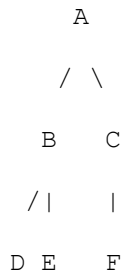
Nontree edges (edges not in BFS tree):

- Undirected graph: all are **cross edges** (connect vertices in different levels).
- Directed graph: can be **back edges** (to ancestors) or cross edges.

BFS traversal time is $O(n + m)$ for adjacency list graphs.

Example BFS Traversal

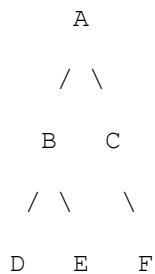
Graph:



- **Start BFS at A:**

- Level 0: A
- Level 1: B, C
- Level 2: D, E, F

BFS Tree:



- Tree edges (solid lines) = discovery edges
- Cross edges (if any) = dashed lines

BFS vs DFS

Feature	BFS	DFS
Traversal order	Level by level	Depth first
Finds shortest path	✓ Yes (min edges)	✗ No guarantee
Data structure	Queue	Stack (recursion)

Connected components	✔ Yes	✔ Yes
Cycle detection	Limited	✔ Better for directed cycles
Use case	Shortest path, social networks	Topological sort, SCC, maze solving

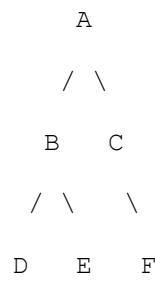
BFS with Levels (Visualization)

Step 0 (Level 0): A

Step 1 (Level 1): B, C

Step 2 (Level 2): D, E, F

BFS Tree:



Applications of BFS

- Shortest path in **unweighted graphs**
- **Social networks** (degrees of separation)
- **Web crawlers** (finding all reachable pages)
- **Connectivity checks** and **connected components**
- AI: **shortest solution paths** in puzzles/mazes